

IENUMERABLE ICOLLECTION IQUERYABLE LIST

WHAT ARE THE DIFFERENCES?



Written by
Petros Sargsian



WHAT IS IENUMERABLE?

IEnumerable is the base interface for all collections. It allows forward-only iteration and executes queries in memory.

- Read-only iteration (foreach)
- LINQ executes in-memory
- Deferred execution
- Cannot add/remove items

```
IEnumerator<Product> products =  
    context.Products.ToList();  
  
var filtered = products  
    .Where(p => p.Price > 100); // In-memory
```



Written by
Petros Sargsian



WHAT IS IQUERYABLE?

IQueryable builds expression trees that translate to SQL. Queries execute on the database server, not in memory.

- Translates LINQ to SQL
- Executes on database
- Deferred execution
- More efficient for large datasets

```
● ● ● +  
  
IQueryable<Product> products =  
    context.Products;  
  
var filtered = products  
    .Where(p => p.Price > 100); // SQL query
```



Written by
Petros Sargsian



WHAT IS ICOLLECTION?

ICollection adds modification capabilities to **IEnumerable**. You can add, remove, and check count without converting to **List**.

- Can add/remove items
- Has Count property
- No indexing (no collection[0])
- Good for navigation properties

```
ICollection<Order> orders =  
    customer.Orders;  
  
orders.Add(newOrder);  
orders.Remove(oldOrder);  
var count = orders.Count;
```



Written by
Petros Sargsian



WHAT IS LIST?

List is a concrete implementation with full functionality. Provides indexing, sorting, searching, and modification.

- Full CRUD operations
- Index access: list[0]
- Sorting and searching
- All data loaded in memory

```
List<Product> products =  
    context.Products.ToList();  
  
products.Sort();  
var first = products[0];  
products.Insert(1, newProduct);
```



Written by
Petros Sargsian



IENUMERABLE VS IQUERYABLE

Key Difference:

- IEnumerable executes **in memory** after loading data.
- IQueryable executes **on the database**.

Performance Impact: IEnumerable loads ALL records then filters. IQueryable filters in SQL.

```
// BAD: Loads 10,000 products to memory
IEnumerable<Product> all = GetProducts();
var filtered = all.Where(p => p.Price > 100);

// GOOD: Database returns only matches
IQueryable<Product> query = GetQuery();
var filtered = query.Where(p => p.Price > 100);
```



Written by
Petros Sargsian



WHEN TO USE EACH

- **IEnumerable**: Read-only, already in memory, simple iteration
- **IQueryable**: Database queries, need SQL optimization, large datasets
- **ICollection**: Navigation properties, need add/remove but not indexing
- **List**: Need full control, sorting, indexing, small datasets

```

// Repository method
IQueryable<Product> GetQuery() { }

// Service layer
IEnumerable<Product> GetFiltered() { }

// Entity navigation
ICollection<Order> Orders { get; set; }
```



Written by
Petros Sargsian



COMMON MISTAKES

- Mistake 1: Using IEnumerable for database queries - loads everything to memory
- Mistake 2: Returning IQueryable from services - leaks database logic
- Mistake 3: Using List when ICollection works - over-specifying

```
// BAD: Loads all products
IEnumerable<Product> GetProducts()
{
    return context.Products; // Executes immediately
}

// GOOD: Query on database
IQueryable<Product> GetProductsQuery()
{
    return context.Products; // Deferred
}
```



Written by
Petros Sargsian





VOILA!

Follow for more content like this.

↻ Repost

+ Follow me